

项目手册：QLoRA finetune LLaMa

目录：

项目手册：QLoRA finetune LLaMa

项目目标

学习方法

一个微调项目的流程

1. 需求分析
2. 模型的选择
3. 找数据
4. 预处理数据
5. 微调以及评估

环境准备

模型下载

安装必要的包

导入数据

配置模型

加载模型

加载tokenizer

测试导入的模型的效果

数据预处理

构造用于 finetune 的模型（模型实例化）

finetune模型

Finetune模型和评估

QLoRA 微调过程中遇到的问题及其解决方案

在这个项目中，我们将会使用qlora微调llama3-8b-instruct模型，qlora量化技术让我们可以使用低成本的GPU进行大模型微调。

预计所需时间：2-4 个小时

项目目标

- 理解整个微调的过程，以及在微调过程中使用到的相关技术
- 理解数据的处理
- 掌握如何对于一个新的问题一步步去解决
- 使用 huggingface models, PEFT 等基础工具。（在后续的项目中，我们会接着去学习其他工具）

学习方法

第一步（必须）：认真阅读每个细节，然后把相关的代码放到 notebook 上去运行并得到结果。

第二步（建议）：尝试自行修改部分代码模块来迎合自己的需求。

注：每一段代码都标注了 Cell 序号，按照顺序把代码粘贴到 jupyter notebook 即可以运行。没有标注 cell 序号的代码是不是必须的。

一个微调项目的流程

1. 需求分析

这个需求就是业务方给到的，比如实现医疗领域的问答系统，或者实现糖尿病领域的医学助理。在这个需求的分析阶段，我们要明确几点：

第一、什么领域的问题？

因为我们并不会考虑从零开始训练模型（假定），所以我们要根据需求找到最适合的大模型底座接着去 finetune。如果要解决金融问题，那我们首先要尝试去寻找金融领域的大模型底座；

第二、大模型需要具备哪方面的能力？

适合金融领域的底座可能会很多，那这时候我们要进一步去观察每个开源模型所具备的能力是否满足我们业务需求。这里有个问题：那我们如何评测这些开源模型是否具备“我们想得到的能力”？答案就是设计一套评测准则：评测数据可以是自己构造的数据，也可以是公开评测数据，或者两者结合的数据。

2. 模型的选择

模型选择一般从几个纬度来考虑：

1. **语言**。使用中文语言模型，还是英文的？这个选择就比较直接了。比如 ChatGLM, Baichuan 就可以支持中文，“经典”的 LLaMA 支持多语言（但还是英文表现最好，中文有待提升）。但即便像 LLaMA 这种模型也有中文版本(<https://github.com/ymcui/Chinese-LLaMA-Alpaca>)。这些中文版本的训练方法一般都是：拿着经典 LLaMA 模型，在这个基础上灌入中文的大量语料库进行 continuous pretrain (也属于预训练过程，花费大量的 GPU 资源)，这样训练出来的大模型就有较好的中文能力，在这个过程中也需要扩充中文词汇 (tokenizer)，因为原来的英文模型里主要以英文的 token 为主。Continuous pretrain 之后，就可以得到 chinese-llama-base 模型，如果在这个基础上进一步做 SFT/SFT+RLHF，即可以得到 chinese-llama-chat model。
2. **选择 Base model 还是 Chat Model?** 这个问题可以从几个方面考虑：

使用 Base 模型的情况：

- 如果我们有大量的专业数据要训练，那这个时候选择 Base 应该会比较合理的，因为我们希望在没有被“污染”过的模型基础上接着训练。Base 模型是最干净的。如果需要对话能力，我们也完全可以引入我们自己的对话数据来驱动大模型的对话能力。（比如做医疗的，那就收集大量医疗相关的对话数据）。
- 不需要对话能力：如果只是追求语言理解能力，我们就可以选择 Base 来接着 finetune。
- 深度定制：比如一个客户要解决金融领域的智能投顾问题，那这个时候从 Base 开始训练也是比较合理的选择。

使用 chat 模型的情况：

- 对于需要理解和生成自然语言对话的应用（如聊天机器人、客户服务自动化、虚拟助手等），

Chat 模型由于其在对话上的专门训练，可能会提供更好的性能。

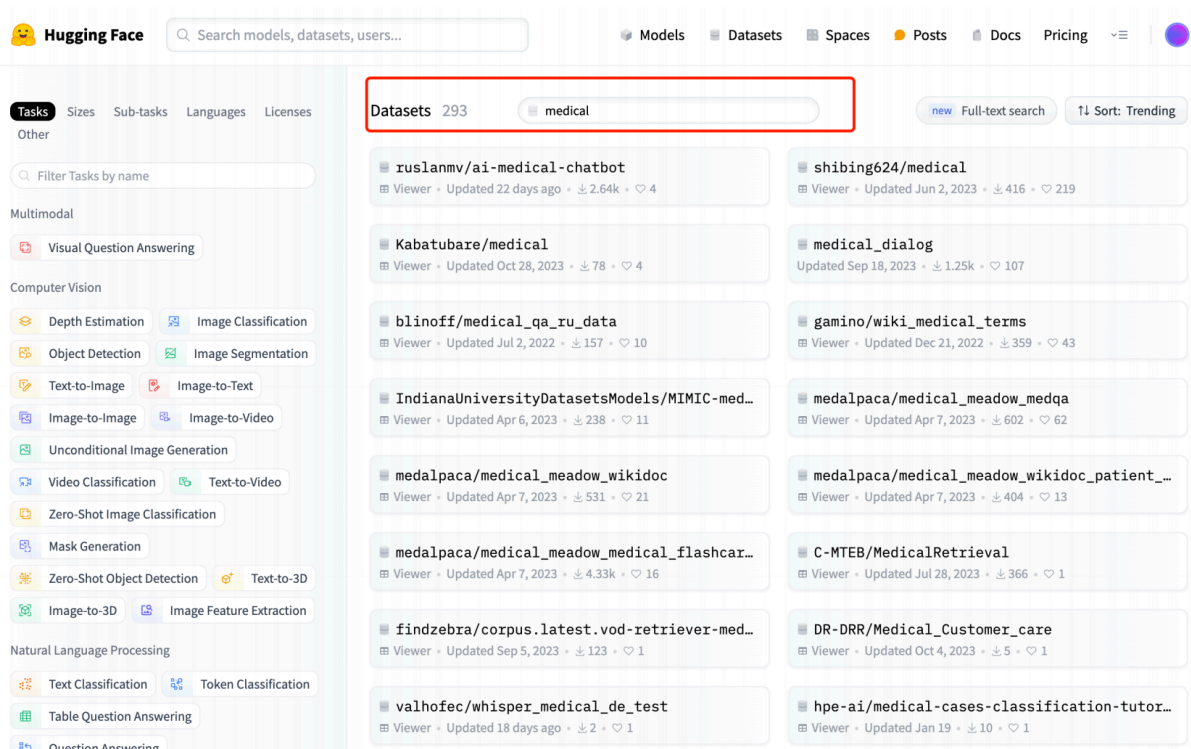
- 如果你的应用场景需要模型理解和维持长时间的对话上下文，Chat 模型通常更适合这种需求。
- 资源有限，可以用 Chat 模型也是比较合理的选择，毕竟想驱动一个大模型的对话能力也是需要对话能力的训练的。

3. 具体选择哪个开源模型？快速的方案是，先列出关注度比较高的开源模型，开源社区的关注度高的模型，通常效果也是可以的。如果做中文，就列出 chatglm, baichuan, Qwen, Chinese-LLaMA 等；如果做英文，那就 LLaMA, mistral 等系列模型可能是首选。接下来，需要进一步考虑能承受的资源限制，还有通过快速的benchmark 验证来判断哪个最适合。

3. 找数据

这个过程会比较费脑筋。有个大家的共识是：“高质量的，少量的 SFT 数据要优于大量的、低质量的 SFT 数据”，所以寻找跟自己业务场景相关的高质量数据非常重要。收集数据有以下几个渠道：

1. 公开数据，比如像在 huggingface 上的，这里有大量的公开数据集供使用。



GitHub 上也有一些好的 repo，整理了一些开源的数据。

下面给大家列举一些

<https://github.com/CLUEbenchmark/CLUEDatasetSearch>

https://github.com/brightmart/nlp_chinese_corpus

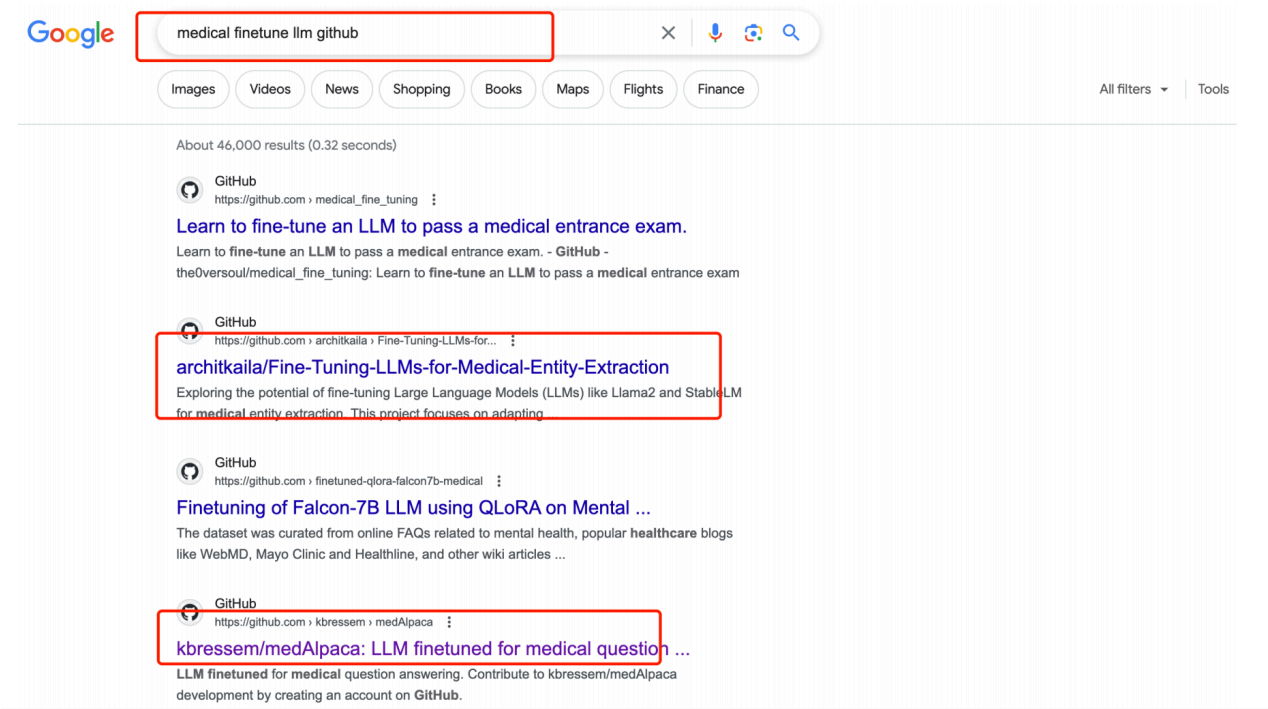
https://github.com/GanjinZero/awesome_Chinese_medical_NLP

<https://github.com/smoothnlp/FinancialDatasets>

<https://github.com/juand-r/entity-recognition-datasets>

<https://github.com/SophonPlus/ChineseNlpCorpus>

跟进一些最新发布的大模型或者领域微调过的大模型，一般也会整理出能用的数据。



总之，想寻找公开能用的数据，需要细心的搜索。

2. **生成数据**：这部分就可以靠 GPT4 等优质的模型来生成。常见的有几种方法：

- 比如给定 100 个已有的 seed 数据，让 GPT4 等优质模型帮我们接着生成其他的类似的数据（像 self-instruct）
- 给定一些文章，把文章内容转换成<input, output>形式。比如一个医学杂志，基于里面的文本，让 GPT4 帮我们说生成一问一答的形式，或者多轮对话形式。举个简单的例子，给定一个文本：“这个感冒药一瓶的价格是 10 块钱”，可以转换成 <input: 这瓶感冒药多少钱? , output: 10 块钱>，这样就构成了 instruction data 格式。同时，也可以扩充成多轮对话，比如让 GPT4 接着帮我们生成问题 然而，GPT4 的价格一般是很贵的，一种这方案是使用 Mistral 等高质量开源大模型来生成，这时候你的花销也就是推理成本。

4. 预处理数据

主要包含两个步骤：

1. 数据的格式化、formatting 等。
2. 字符串数据转换成大模型能理解的 token_ids 系列数据，这部分主要由 tokenizer 来完成。

5. 微调以及评估

微调过程也有几个不同的选择，比如全量微调（full fine-tune），高效微调（PEFT）。

如果是全量微调，其实跟预训练的训练方式是一样的，只不过微调阶段数据量会少很多。一般我们建议是 PEFT，一方面节省资源，另外一方面可以提高模型的泛化能力（generalization）。

一般常用的有 LoRA，QLoRA 等方法。如果对这些方法不太熟悉，可以看看课程回放视频。

接下来，我们会正式使用 Qlora 方法来微调 llama3-7b-instruct 模型。

项目流程

1. 搭建环境和安装必要的包（library）
2. 模型下载
3. 导入数据
4. 对于模型进行配置
5. 导入 Pre-trained model
6. 导入 tokenizer
7. 测试导入进来的模型
8. 对于数据进行预处理，符合我们模型训练所需要的输入
9. 进行训练前的配置（QLoRA）
10. 配置 PEFT 模型
11. 进行 finetune 训练
12. 加载finetune后的模型进行推理

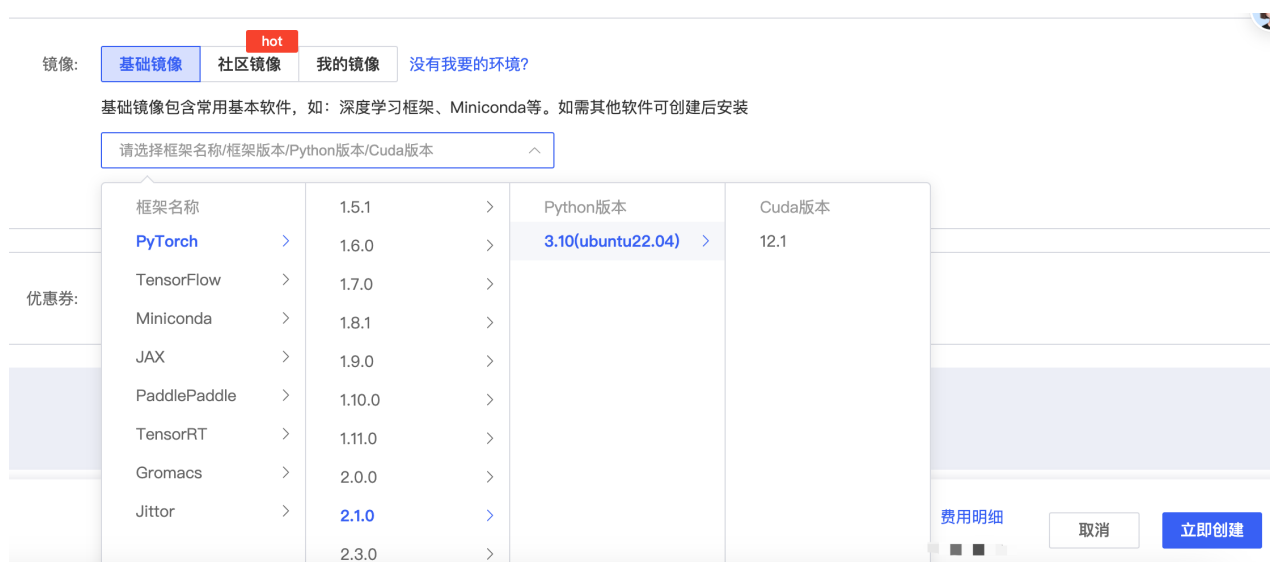
环境准备

如果没有自己的GPU机器，可以选择租赁一台云GPU，市面上有很多供大家选择的，如阿里云，百度云，google colab等等，这里使用AutoDL作为示例讲解。

在 Autodl 平台中租赁一个 3090 等 24G 显存的显卡机器，在镜像选择中选择基础镜像：

```
PyTorch-->2.1.0-->3.10(ubuntu22.04)-->12.1
```

接下来打开刚刚租用服务器的 JupyterLab，并且打开其中的终端开始环境配置、模型下载和进行实验。



模型下载

在开始代码之前，我们先下载模型。

我们使用 modelscope 中的 snapshot_download 函数下载模型，第一个参数为模型名称，参数 cache_dir 为模型的下载路径。

先安装下modelscope包：

```
pip install modelscope
```

在 /root/autodl-tmp 路径下新建 model_download.py 文件并在其中输入以下内容，粘贴代码后记得保存文件，如下图所示。并运行 `python /root/autodl-tmp/model_download.py` 执行下载，模型大小为 15 GB，下载模型大概需要 2 分钟。

```
import torch
from modelscope import snapshot_download, AutoModel, AutoTokenizer
import os

model_dir = snapshot_download('LLM-Research/Meta-Llama-3-8B-Instruct',
cache_dir='/root/autodl-tmp', revision='master')
```

下载好的模型目录如下：

```
root@autodl-container-d577448475-cad06feb:~/autodl-tmp/LLM-Research/Meta-Llama-3-8B-Instruct# ll
total 15693184
drwxr-xr-x 2 root root      4096 Jun 11 22:14 ./
drwxr-xr-x 3 root root       38 Jun 11 22:08 ../
-rw-r--r-- 1 root root       60 Jun 11 22:08 .mdl
-rw----- 1 root root    1080 Jun 11 22:14 .msc
-rw-r--r-- 1 root root       36 Jun 11 22:14 .mv
-rw-r--r-- 1 root root     7801 Jun 11 22:08 LICENSE
-rw-r--r-- 1 root root    37204 Jun 11 22:14 README.md
-rw-r--r-- 1 root root     4696 Jun 11 22:14 USE_POLICY.md
-rw-r--r-- 1 root root      654 Jun 11 22:08 config.json
-rw-r--r-- 1 root root      48 Jun 11 22:08 configuration.json
-rw-r--r-- 1 root root     187 Jun 11 22:08 generation_config.json
-rw-r--r-- 1 root root 4976698672 Jun 11 22:09 model-00001-of-00004.safetensors
-rw-r--r-- 1 root root 4999802720 Jun 11 22:11 model-00002-of-00004.safetensors
-rw-r--r-- 1 root root 4915916176 Jun 11 22:13 model-00003-of-00004.safetensors
-rw-r--r-- 1 root root 1168138808 Jun 11 22:14 model-00004-of-00004.safetensors
-rw-r--r-- 1 root root     23950 Jun 11 22:14 model.safetensors.index.json
-rw-r--r-- 1 root root       73 Jun 11 22:14 special_tokens_map.json
-rw-r--r-- 1 root root   9085698 Jun 11 22:14 tokenizer.json
-rw-r--r-- 1 root root     50982 Jun 11 22:14 tokenizer_config.json
```

安装必要的包

【cell1】

```
!pip install datasets peft bitsandbytes transformers trl pynvml
```

需要以下几个包：

1. transformers： 它提供了许多预训练的模型，这些模型可以用于自然语言处理（NLP）任务，比如文本分类、信息抽取、问答、摘要、翻译以及其他与语言相关的任务。它支持 BERT、GPT、T5、XLNet 等多种模型架构。注意：模型架构和模型本身是不一样的，比如 GPT4 是基于 GPT 的模型架构设计的，LLaMA 也是基于 GPT 的模型架构设计的。
2. peft： 用于 efficient fine tuning, 比如 Lora 等
3. bitsandbytes

专注于提供高效的位操作和字节级处理功能，尤其适用于深度学习和自然语言处理任务中对性能要求较高的场景。这个库提供了一些底层操作，可以加速数据处理，尤其是在需要进行位打包、解包或进行位级逻辑运算时。

4. datasets: datasets 是 Hugging Face 提供的一个轻量级、易于使用、可扩展的 NLP 数据集库。它提供了快速访问的接口，可以下载和处理多种 NLP 任务所需的数据集。
5. trl: trl 是“transformer-reinforcement learning”，这是一个结合了变换器（transformers）模型和强化学习（reinforcement learning）的库。它通常用于优化语言模型的生成性能，主要用于 SFT, reward model, 强化学习。

安装好相应地包后，需要导入相应地模块：

【cell2】

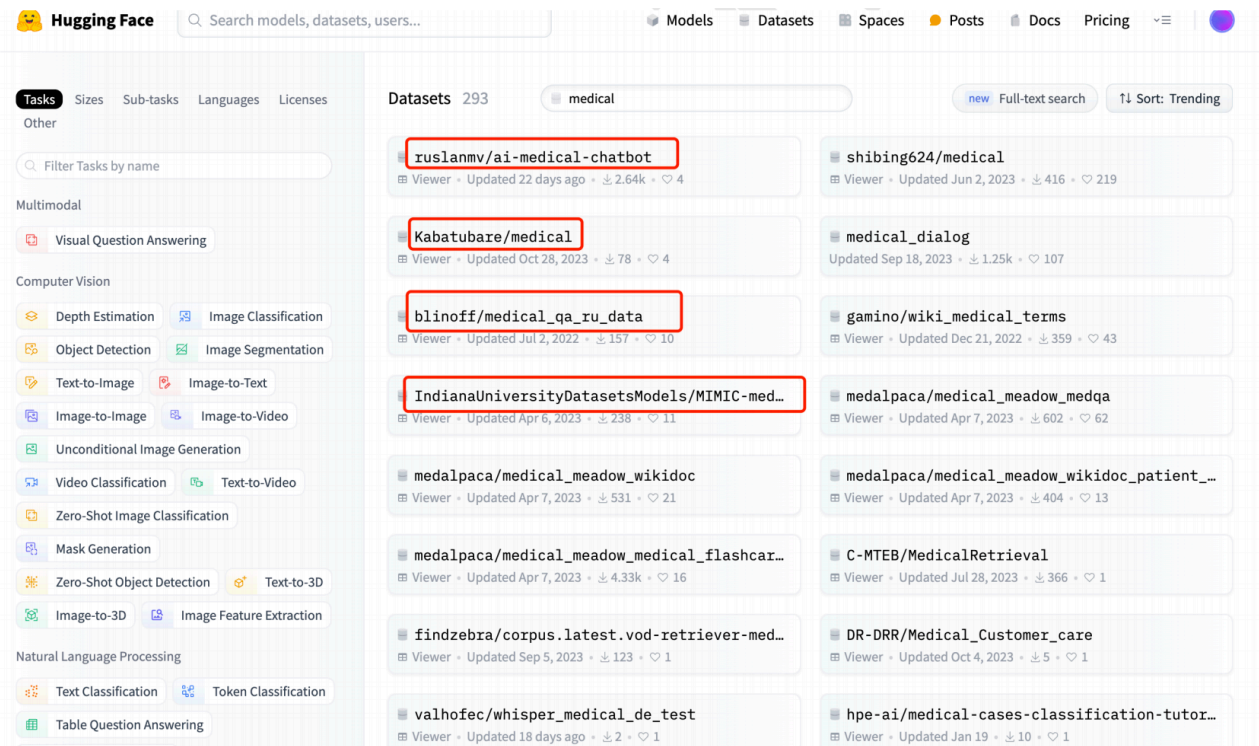
```
import os
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    TrainingArguments,
    pipeline,
    logging,
)
from peft import LoraConfig
from trl import SFTTrainer
from tqdm import tqdm
import torch
import time
import pandas as pd
import numpy as np
```

详细的解释：

```
from datasets import load_dataset
```

load_dataset 函数用于方便地下载或加载数据集。你只需要指定相应的数据集名，就可以把数据集读进来。

下面标注为红色的部分为数据集名，作为参数即可。



from transformers import ... 代码从 transformers 库中导入了几个类和配置：

- AutoModelForCausalLM：用于自动加载或下载因果语言模型（如 GPT 系列），这类模型适用于生成任务。
- AutoTokenizer：用于自动加载或下载与特定模型相对应的 tokenizer。
- BitsAndBytesConfig：这通常是与 bitsandbytes 库相关的配置，用于设置模型训练时的低精度优化参数。
- TrainingArguments：包含了训练模型时所需的各种参数。

几个有用的命令和函数解释：

在命令行中输入 `nvidia-smi` 或者在 jupyter notebook 中输入 `!nvidia-smi`，可以观察 gpu 的占用情况。

```
: !nvidia-smi
```

```
Tue Jun 11 23:44:01 2024
```

NVIDIA-SMI 550.67		Driver Version: 550.67				CUDA Version: 12.4			
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC				
Fan	Temp	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.				
	Perf				MIG M.				
0	NVIDIA GeForce RTX 3090	On	00000000:52:00:0	Off	N/A				
30%	36C P8	21W / 350W	4MiB / 24576MiB	0%	Default				
					N/A				

Processes:							GPU Memory Usage
GPU	GI	CI	PID	Type	Process name		
	ID	ID					
No running processes found							

导入数据

在导入数据之前，如果你使用的是AutoDL平台，需要配置一下代理以便我们可以顺利访问到huggingface。

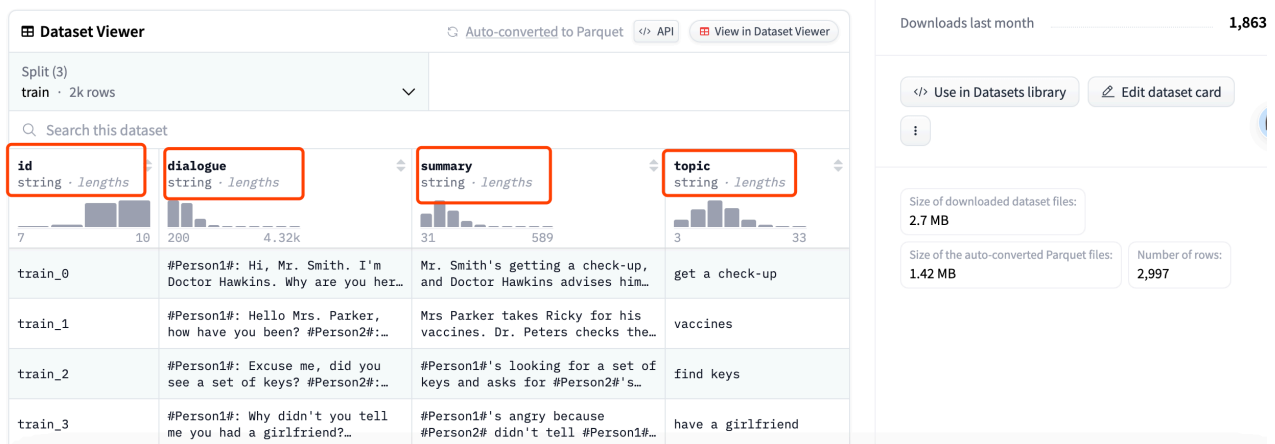
```
import subprocess
import os

result = subprocess.run('bash -c "source /etc/network_turbo && env | grep proxy"', shell=True, capture_output=True, text=True)
output = result.stdout
for line in output.splitlines():
    if '=' in line:
        var, value = line.split('=', 1)
        os.environ[var] = value
```

这里我们将使用一个英文的 summarization 数据。在 huggingface 上的地址为：

<https://huggingface.co/datasets/neil-code/dialogsum-test>

打开网页之后，可以看到如下的页面：



数据集已经被 split 成 train, validation, test。

Id: 字段就是数据的 id

Dialogue: 字段就是多轮对话

Summary: 字段是对于对轮对话的总结

所以这个任务，其实是根据给定的 dialogue，生成对话的 summary。

另外，huggingface 存储文件使用的是 version control, 所以获取数据的时候，也要关注具体的 version, 就像我们使用 git 一样。具体数据文件：

Dataset card	Viewer	Files and versions	Community
--------------	--------	--------------------	-----------

main	dialogsum-test	1 contributor	History: 2 commits	+ Contribute
------	----------------	---------------	--------------------	--------------

neil-code	Upload 6 files	f0524dd	10 months ago
.gitattributes	2.31 kB	initial commit	10 months ago
README.md	4.56 kB	Upload 6 files	10 months ago
cut.py	1.36 kB	Upload 6 files	10 months ago
holdout.csv	86.4 kB	Upload 6 files	10 months ago
test.csv	447 kB	Upload 6 files	10 months ago
train.csv	1.81 MB	Upload 6 files	10 months ago
validation.csv	441 kB	Upload 6 files	10 months ago

默认是 main 分支，如果改变为其他的分支，需要在读取数据时指定相应的分支。

读取上面的数据，使用如下的代码：

【cell3】

```
dataset_name = "neil-code/dialogsum-test"
dataset = load_dataset(dataset_name)
```

读取完之后，可以观察一下 dataset 里面存放的数据以及格式。

dataset

```
DatasetDict({
  train: Dataset({
    features: ['id', 'dialogue', 'summary', 'topic'],
    num_rows: 1999
  })
  validation: Dataset({
    features: ['id', 'dialogue', 'summary', 'topic'],
    num_rows: 499
  })
  test: Dataset({
    features: ['id', 'dialogue', 'summary', 'topic'],
    num_rows: 499
  })
})
```

观察 dataset 结构，可以发现里面有三个数据，分别是 train, validation, test，而且读进来的字段跟 huggingface 上看到的字段是一样的。

如果看某一个具体的数据：

```
dataset['train'][0]
```

```
{'id': 'train_0',  
'dialogue': '"#Person1#: Hi, Mr. Smith. I'm Doctor Hawkins. Why are you here today?\n#Person2#: I found it would be a  
good idea to get a check-up.\n#Person1#: Yes, well, you haven't had one for 5 years. You should have one every yea  
r.\n#Person2#: I know. I figure as long as there is nothing wrong, why go see the doctor?\n#Person1#: Well, the best  
way to avoid serious illnesses is to find out about them early. So try to come at least once a year for your own goo  
d.\n#Person2#: Ok.\n#Person1#: Let me see here. Your eyes and ears look fine. Take a deep breath, please. Do you smok  
e, Mr. Smith?\n#Person2#: Yes.\n#Person1#: Smoking is the leading cause of lung cancer and heart disease, you know. Y  
ou really should quit.\n#Person2#: I've tried hundreds of times, but I just can't seem to kick the habit.\n#Person1#: Well, we have classes and some medications that might help. I'll give you more information before you leave.\n#Person  
2#: Ok, thanks doctor."',  
'summary': '"Mr. Smith's getting a check-up, and Doctor Hawkins advises him to have one every year. Hawkins'll give s  
ome information about their classes and medications to help Mr. Smith quit smoking."',  
'topic': 'get a check-up'}
```

有两个字段我们需要用到：dialogue 和 summary。一个作为输入，另外一个作为输出。但我们要 finetune 的是 LLM，格式是要满足 instruction data 的格式。如果是传统 NLP 任务，我们实际上可以把<dialogue, summary>直接作为输入和输出送到模型里来处理。

但这里我们需要构造 instruction data。构造过程其实也简单，如设计这种 prompt：

```
Give the conversation, extract the main points and summarize the conversions  
{dialogue}  
{summary}
```

这部分可以通过编写一个函数来实现对每一个输入数据的formatting。

【cell4】

```
def create_prompt_formats(sample):  
    """  
    Format various fields of the sample ('instruction','output')  
    :param sample: input data  
    """  
  
    INTRO_BLURB = "Instruct: Below is an instruction that describes a task.  
Write a response that appropriately completes the request."  
    INSTRUCTION_KEY = "Input: Please Summarize the below conversation."  
    RESPONSE_KEY = "Output:"  
  
    blurb = f"\n{INTRO_BLURB}"  
    instruction = f"{INSTRUCTION_KEY}"  
    input_context = f"{sample['dialogue']}" if sample["dialogue"] else None  
    response = f"{RESPONSE_KEY}\n{sample['summary']}"  
  
    parts = [part for part in [blurb, instruction, input_context, response] if  
part]  
    formatted_prompt = "\n\n".join(parts)  
    sample["text"] = formatted_prompt  
    return sample
```

我们可以调用下改写代码查看一下改写后的格式

```
print(create_prompt_formats(dataset['train'][0])['text'])
```

```
print(create_prompt_formats(dataset['train'][0])['text'])
```

Instruct: Below is an instruction that describes a task. Write a response that appropriately completes the request.

Input: Please Summarize the below conversation.

```
#Person1#: Hi, Mr. Smith. I'm Doctor Hawkins. Why are you here today?
#Person2#: I found it would be a good idea to get a check-up.
#Person1#: Yes, well, you haven't had one for 5 years. You should have one every year.
#Person2#: I know. I figure as long as there is nothing wrong, why go see the doctor?
#Person1#: Well, the best way to avoid serious illnesses is to find out about them early. So try to come at least once a year for your own good.
#Person2#: Ok.
#Person1#: Let me see here. Your eyes and ears look fine. Take a deep breath, please. Do you smoke, Mr. Smith?
#Person2#: Yes.
#Person1#: Smoking is the leading cause of lung cancer and heart disease, you know. You really should quit.
#Person2#: I've tried hundreds of times, but I just can't seem to kick the habit.
#Person1#: Well, we have classes and some medications that might help. I'll give you more information before you leave.
#Person2#: Ok, thanks doctor.
```

Output:

Mr. Smith's getting a check-up, and Doctor Hawkins advises him to have one every year. Hawkins'll give some information about their classes and medications to help Mr. Smith quit smoking.

可以看到Instruct, Input和Output部分都正确提取到了。

我们调用 `create_prompt_formats` 的时候加了一个新的字段“text”，并把改好的 prompt 和 output 一起写到“text”字段里。这时候，其他的字段就不太需要了，如“id”，“dialogue”，“summary”等，这些后续我们需要去掉。

```
print(create_prompt_formats(dataset['train'][0]))
```

```
{'id': 'train_0', 'dialogue': '#Person1#: Hi, Mr. Smith. I'm Doctor Hawkins. Why are you here today?\n#Person2#: I found it would be a good idea to get a check-up.\n#Person1#: Yes, well, you haven't had one for 5 years. You should have one every year.\n#Person2#: I know. I figure as long as there is nothing wrong, why go see the doctor?\n#Person1#: Well, the best way to avoid serious illnesses is to find out about them early. So try to come at least once a year for your own good.\n#Person2#: Ok.\n#Person1#: Let me see here. Your eyes and ears look fine. Take a deep breath, please. Do you smoke, Mr. Smith?\n#Person2#: Yes.\n#Person1#: Smoking is the leading cause of lung cancer and heart disease, you know. You really should quit.\n#Person2#: I've tried hundreds of times, but I just can't seem to kick the habit.\n#Person1#: Well, we have classes and some medications that might help. I'll give you more information before you leave.\n#Person2#: Ok, thanks doctor.', 'summary': 'Mr. Smith's getting a check-up, and Doctor Hawkins advises him to have one every year. Hawkins'll give some information about their classes and medications to help Mr. Smith quit smoking.', 'topic': 'get a check-up', 'text': '\nInstruct: Below is an instruction that describes a task. Write a response that appropriately completes the request.\n\nInput: Please Summarize the below conversation.\n\n#Person1#: Hi, Mr. Smith. I'm Doctor Hawkins. Why are you here today?\n#Person2#: I found it would be a good idea to get a check-up.\n#Person1#: Yes, well, you haven't had one for 5 years. You should have one every year.\n#Person2#: I know. I figure as long as there is nothing wrong, why go see the doctor?\n#Person1#: Well, the best way to avoid serious illnesses is to find out about them early. So try to come at least once a year for your own good.\n#Person2#: Ok.\n#Person1#: Let me see here. Your eyes and ears look fine. Take a deep breath, please. Do you smoke, Mr. Smith?\n#Person2#: Yes.\n#Person1#: Smoking is the leading cause of lung cancer and heart disease, you know. You really should quit.\n#Person2#: I've tried hundreds of times, but I just can't seem to kick the habit.\n#Person1#: Well, we have classes and some medications that might help. I'll give you more information before you leave.\n#Person2#: Ok, thanks doctor.\n\nOutput:\nMr. Smith's getting a check-up, and Doctor Hawkins advises him to have one every year. Hawkins'll give some information about their classes and medications to help Mr. Smith quit smoking.'}
```

配置模型

在导入模型之前，我们需要设置一些Lora和量化的配置参数。

因为我们在这个项目中要使用 QLoRA，而且要 quantize 成 4bit，所以这些信息要提前做好配置。这样模型导入的时候，就会被按照我们的要求转换为 4bit 的量化模型。

```
compute_dtype = getattr(torch, "float16")

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=compute_dtype,
    bnb_4bit_use_double_quant=True,
)
```

这里解释一下每个参数：

1. `compute_dtype = getattr(torch, "float16")`:
 - 这行代码使用 `getattr` 函数从 PyTorch 库中获取 `float16` 数据类型。`float16` 是 16 位浮点数，它具有较小的数值范围和精度，但比 32 位浮点数（`float32`）占用更少的内存和计算资源。在量化过程中，使用 `float16` 可以减少模型的内存占用和加速计算。
2. `quant_config = BitsAndBytesConfig(...)`:
 - `BitsAndBytesConfig` 是 `BitsAndBytes` 库中用于配置量化参数的类。
3. `load_in_4bit=True`:
 - 这个参数指定模型在加载时是否使用 4 位量化。设置为 `True` 意味着模型在加载时会进行 4 位量化处理。
4. `bnb_4bit_quant_type="nf4"`:
 - 这个参数定义了 4 位量化的类型。`"nf4"` 代表 "non-functional 4-bit" 量化，这是一种非功能性的量化方法，通常用于测试和实验，因为它不会改变模型的权重值。
5. `bnb_4bit_compute_dtype=compute_dtype`:
 - 这个参数设置了在 4 位量化计算时使用的数值类型。这里使用了之前定义的 `compute_dtype`，即 `float16`。这意味着在进行 4 位量化计算时，会使用 16 位浮点数来处理数值。
6. `bnb_4bit_use_double_quant=True`:
 - 这个参数指定是否使用双重量化。双重量化是一种技术，可以在不损失太多精度的情况下进一步减少模型大小。双重量化是为了进一步量化 quantization constant，也是 Qlora 里面提出的一种方式。

加载模型

【cell5】

```
model_path = "/root/autodl-tmp/LLM-Research/Meta-Llama-3-8B-Instruct/"
original_model = AutoModelForCausalLM.from_pretrained(
    model_path,
    torch_dtype=compute_dtype,
    device_map={"": 0},
    quantization_config=quant_config
)
```

下面对参数做一些解释：

- `model_path`:这里我们指定了提前下载好的模型的本地路径，当然如果网络通畅的话，也可以将 `model_path` 替换为 `huggingface` 上的模型名字，这样可以自动下载到本地。
- `device_map`: 用于定义模型参数应该放置在哪个计算卡上。字典的键是模型的参数名前缀，值是设备的编号。在这个例子中，键是一个空字符串""，它通常代表所有的模型参数，而值 0 通常代表第一个可用的 GPU。这意味着所有模型参数都应该放在编号为 0 的 GPU 上。如果你有多个 GPU，可以通过这个映射来指定不同的参数应该放在哪个 GPU 上，以实现模型的并行计算。
- `quantization_config=quant_config`: 使用 4 位量化配置，意味着模型在加载时使用这些量化设置，以优化内存使用和可能的计算效率。

加载之后可以使用 `!nvidia-smi` 查看一下内存占用：

`!nvidia-smi`

Mon Jun 17 21:45:54 2024

NVIDIA-SMI 550.54.14				Driver Version: 550.54.14				CUDA Version: 12.4			
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC				
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.	MIG M.				
0	NVIDIA GeForce RTX 3090	On	00000000:4F:00.0	Off			N/A				
30%	27C	P2	113W / 350W	5882MiB / 24576MiB	0%	Default	N/A				

Processes:

GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
ID	ID	ID				

只占用了5G多一些，可见量化确实是可以节约显存的。

加载tokenizer

Tokenizer 的作用为把字符串形式的文字转化为 token 的 id, 这是必要的过程。比如一句话“大模型的发展很快”，被指定的 tokenizer 分之后变成“大 模 型 的 发 展 很 快”，之后每一个 token 会被映射到这个 token 的 index，比如 [76, 199, 300, 12, 567, 1142, 8775]。

所以我们需要导入适合当前模型的 tokenizer，并进行后续 tokenize 的操作。

【cell6】

```
tokenizer = AutoTokenizer.from_pretrained(model_path, use_fast=False,
trust_remote_code=True,
padding_side="left", add_eos_token=True, add_bos_token=True)
tokenizer.pad_token_id = tokenizer.eos_token_id
```

- `model_name` 作为参数，会自动选择跟 `model_name` 适配的 tokenizer。

- padding_side=left 的意思是，这个参数指定了在序列长度不足时，在序列的哪一侧添加填充（padding）。在这里设置为"left"意味着填充将被添加到序列的左侧。比如我们在进行 mini-batch 训练的时候，为了达到最好的训练效率，会把 mini-batch 里面的 input 长度弄成一样的。一般的操作是在 mini-batch 中，假如有 10 个不同的 input，而且每个 input 长度不一样，这时候可以选择最长的作为标准，对于剩下的 input，长度不足的部分用 padding token 来填充，可以在序列的右边添加，也可以在左边添加。
- add_eos_token=True: 这个参数指示分词器在序列的末尾添加一个结束符（EOS, End Of Sentence）。add_bos_token=True: 这个参数指示分词器在序列的开头添加一个开始符（BOS, Beginning Of Sentence）。比如一句话"I love this cat", 添加完之后变成 `<bos>I love this cat<eos>`。
- use_fast=False: 这个参数指定是否使用分词器的快速版本。设置为 False 意味着将使用分词器的标准版本，这通常是基于 Python 的。快速分词器通常是基于 Rust 编写的，提供更好的性能和额外的功能，但在某些情况下可能不支持某些特殊的自定义行为。
- tokenizer.pad_token = tokenizer.eos_token: 这行代码将分词器的填充令牌（pad_token）设置为与结束符令牌（eos_token）相同。这是必要的。
- 到目前为止，把相应的模型和 tokenizer，以及数据导入进来了。接下来我们可以简单测试一下模型的效果（就是 fine-tune 之前的）

测试导入的模型的效果

测试过程实际上是对模型的 inference 过程（推理过程）。需要做：

1. 构造模型的输入：这部分就是通过 template 来构造即可，结果发送给模型
2. 根据输入调用模型，并生成输出，最后再通过 tokenizer 把输出转化为字符串的形式（用户能看得懂的形式）

整体的推理代码为：

【cell7】

```
eval_tokenizer = AutoTokenizer.from_pretrained(model_path, add_bos_token=True,
trust_remote_code=True, use_fast=False)
eval_tokenizer.pad_token = eval_tokenizer.eos_token
def gen(model,p, maxlen=100, sample=True):
    toks = eval_tokenizer(p, return_tensors="pt")
    res = model.generate(**toks.to("cuda"), max_new_tokens=maxlen,
do_sample=sample,num_return_sequences=1,temperature=0.1,num_beams=1,top_p=0.95,
).to('cpu')
    return eval_tokenizer.batch_decode(res,skip_special_tokens=True)

%%time
from transformers import set_seed
seed = 42
set_seed(seed)
index = 10

prompt = dataset['test'][index]['dialogue']
```

```

summary = dataset['test'][index]['summary']
formatted_prompt = f"Instruct: Summarize the following conversation.\nInput:
{prompt}\nOutput:\n"
res = gen(original_model,formatted_prompt,100,)
#print(res[0])
output = res[0].split('Output:\n')[1]
dash_line = '-'.join(' ' for x in range(100))
print(dash_line)
print(f'INPUT PROMPT:\n{formatted_prompt}')
print(dash_line)
print(f'BASELINE HUMAN SUMMARY:\n{summary}\n')
print(dash_line)
print(f'MODEL GENERATION - ZERO SHOT:\n{output}')

```

也就是抽取“test”的数据中的其中一个样本进行测试，并观察最终的输出是什么。核心的代码是先构建 formatted_prompt, 之后调用 llama 模型获取 res, 并从 res 里抽取 output 的部分来输出。

这里我们定义 eval_tokenizer, 相比用于训练的 tokenizer 有细微的差异, 比如在参数上 padding=left 没有设置, 其他的基本差不多, 不再做解释。

我们重点来看一下 gen()函数。

`toks = eval_tokenizer(p, return_tensors="pt")`, 这里的 p 指的是输入的 prompt, 是文字的形式, 需要把它转换为 token ids 的形式。return_tensors="pt"指的是分词器返回的编码（比如输入文本的数字表示）将被转换为 PyTorch 的 torch.Tensor 对象。

`res = model.generate(...)`: 使用模型的 generate 方法生成文本。此方法接收以下参数:

- `**toks.to("cuda")`: 将编码后的提示张量移至 GPU 以进行加速计算。
- `max_new_tokens=maxlen`: 生成的最大 token 数。
- `do_sample=sample`: 是否在生成时采样, 如果为 True, 则在生成时使用概率分布, 这使得生成的文本更多样化。
- `num_return_sequences=1`: 生成的序列数量。
- `temperature=0.1`: 控制生成过程中随机性的温度参数, 较低的值 (如 0.1) 会导致更确定性的 (也就是说, 更少随机性的) 输出。
- `num_beams=1`: 使用的 beam search 的 beam 数量, 设置为 1 意味着不使用 beam search。
- `top_p=0.95`: 使用 nucleus sampling 时保留的累积概率分布的部分。比如按照单词分布的大小排序之后: 单词 1: 概率 1, 单词 2: 概率 2, 单词 3: 概率 3 ... 如果概率 1+概率 2+...概率 20, 刚好 ≥ 0.95 , 那我们就从前 20 个单词中进行采样。

`.to('cpu')`: 将生成的文本张量移回 CPU。

`return eval_tokenizer.batch_decode(res, skip_special_tokens=True)`: 使用分词器将生成的张量解码为字符串, 这样用户就可以看得懂了!

把上述代码运行之后就可以得到如下的 output

INPUT PROMPT:

Instruct: Summarize the following conversation.

Input:#Person1#: Happy Birthday, this is for you, Brian.

#Person2#: I'm so happy you remember, please come in and enjoy the party. Everyone's here, I'm sure you have a good time.

#Person1#: Brian, may I have a pleasure to have a dance with you?

#Person2#: Ok.

#Person1#: This is really wonderful party.

#Person2#: Yes, you are always popular with everyone. and you look very pretty today.

#Person1#: Thanks, that's very kind of you to say. I hope my necklace goes with my dress, and they both make me look good I feel.

#Person2#: You look great, you are absolutely glowing.

#Person1#: Thanks, this is a fine party. We should have a drink together to celebrate your birthday

Output:

BASELINE HUMAN SUMMARY:

#Person1# attends Brian's birthday party. Brian thinks #Person1# looks great and charming.

MODEL GENERATION - ZERO SHOT:

The conversation is about a birthday party. Person1, who is Brian, is given a gift and invited to the party. Person2 is the birthday person. They dance together and Person2 compliments Person1 on their appearance. Person1 thanks them and suggests having a drink together to celebrate the birthday. The party is described as wonderful and the atmosphere is festive.

CPU times: user 4.04 s, sys: 134 ms, total: 4.17 s

Wall time: 4.16 s

可以看到模型生成的总结和人类的总结还是有一定差距的，而且搞混了Person1和Brian。

接下来，我们讲开始着手 fine-tune 模型。之前，唯一需要再做的就是数据的预处理。

需要把所有的训练数据进行预处理，使得能够进行训练。

数据预处理

预处理主要分为两个部分：

1. 对于给定的数据，进行格式化，使得满足 instruction 数据所需要的格式。
2. 把文本转换成 token ids 的形式

【cell8】

```
def get_max_length(model):
    conf = model.config
    max_length = None
    for length_setting in ["n_positions", "max_position_embeddings",
                           "seq_length"]:
        max_length = getattr(model.config, length_setting, None)
        if max_length:
            print(f"Found max length: {max_length}")
            break
    if not max_length:
        max_length = 1024
    print(f"Using default max length: {max_length}")
    return max_length

def preprocess_batch(batch, tokenizer, max_length):
    """
    Tokenizing a batch
    """
```

```

return tokenizer(
    batch["text"],
    max_length=max_length,
    truncation=True,
)

```

【cell9】

```

from functools import partial
def preprocess_dataset(tokenizer: AutoTokenizer, max_length: int, seed,
dataset):
    # Add prompt to each sample
    print("Preprocessing dataset...")
    dataset = dataset.map(create_prompt_formats)
    _preprocessing_function = partial(preprocess_batch, max_length=max_length,
tokenizer=tokenizer)
    dataset = dataset.map(
        _preprocessing_function,
        batched=True,
        remove_columns=['id', 'topic', 'dialogue', 'summary'],
    )
    # 过滤样本
    dataset = dataset.filter(lambda sample: len(sample["input_ids"]) <
max_length)
    # Shuffle 数据
    dataset = dataset.shuffle(seed=seed)
    print("Preprocessing dataset done.")
    return dataset

```

其中，几个函数重点解释一下：

1. get_max_length 函数：

- 这个函数检查模型配置，尝试找到模型可以处理的最大序列长度。
- 它检查模型配置中的几个关键字（如"n_positions", "max_position_embeddings", "seq_length"）来确定最大长度。
- 如果找不到这些字段，则默认使用长度 1024。

2. preprocess_batch 函数：

- 这个函数负责将文本转换成 id 的形式（利用 tokenizer）
- 它使用分词器将 batch 中的文本转换为模型可以理解的格式，同时确保序列长度不超过最大长度，如果超过则截断(truncation=True)。

3. preprocess_dataset 函数：

- 它首先调用 dataset.map(create_prompt_formats)来格式化数据集中的每个样本。
- 然后，它定义了一个预处理函数（使用 partial），移除不再需要的列。
- 接着，它使用 dataset.filter 来移除任何超过最大长度限制的样本。
- 最后，它使用 dataset.shuffle 来打乱数据集，确保数据的随机性。

有了以上的函数之后，我们就可以调用下面的代码来创建训练时所需要的数据形式：

【cell10】

```
max_length = get_max_length(original_model)
print(max_length)
train_dataset = preprocess_dataset(tokenizer, max_length, seed,
dataset['train'])
eval_dataset = preprocess_dataset(tokenizer, max_length, seed,
dataset['validation'])
```

通过以上的代码就可以获取 train_dataset 和 eval_dataset。train_dataset 输出之后如下格式：

```
: train_dataset

: Dataset({
    features: ['text', 'input_ids', 'attention_mask'],
    num_rows: 1999
})
```

其中，input_ids 为具体每个 token 的 ids，attention_mask 这里默认都为 1。

多个 instruction data 处理：

处理方式跟单个 instruction data 时没有本质区别，都要经过上述的过程，如 prompt 的标准话，再基于 tokenizer 转化为 ids 的形式。但这里可能有一些点需要考虑：

- 每一种 instruction data 同等重要：这时候可以把所有的 instruction data（来自不同数据源的）做 shuffle，之后按部就班做 mini-batch training 即可。
- 每一种 instruction data 重要性不一样，那这时候我们可能把更高的权重给到更重要的 instruction data，就意味着这些 instruction data 被选取的概率更高。

多个 attention_mask 处理：

在这个项目中，为了简单起见，我们并没有特殊去处理这部分的内容。但是在大多数 fine-tune 的过程中，我们需要留意一下这个问题。你也可以试着改写这部分来适应下面的改动。举个例子，假如有一个样本：

用户：我今天想要参加一个活动

AI：什么活动啊？

对于模型来讲，它的输入实际上为：“用户：我今天想要参加一个活动, AI: 什么活动啊？”，也就是用户和 AI 的对话合在一起。在 fine-tune 阶段我们重点考察的是给定用户的问题，考察 AI 的回复能力。

回顾一下模型的训练过程，它是给定之前的上下文来预测下一个单词，以此类推。所以给定 input 的时候，从第一个单词开始去预测。但在我们情况下有所不通，我们关注的是 AI 的回复，所以一种更实用的处理方式是，在计算训练 loss 的时候只关注 AI 的回复，而不对于用户的问题进行 loss 的计算。

构造用于 finetune 的模型（模型实例化）

接下来根据配置来构造用于训练的模型。什么叫用于训练的模型？我们刚刚 load 进来的模型是预训练模型，是已经在大量的数据上训练的。接下来，我们要在它的基础上做 finetune，这就意味着我们不会改变预训练模型本身的参数，而是训练一个新的模块来为下游任务进行调整。原来模型的参数为 W ，则新的模型参数为 ΔW ，那为了训练 ΔW ，我们需要一个新的模型，包含相应的参数，同时我们要把原来的参数设置为不可训练。通过 `get_peft_model` 就可以构造此模型。

【cell11】

```
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
config = LoraConfig(
    r=32, #Rank
    lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    bias="none",
    lora_dropout=0.01, # Conventional
    task_type="CAUSAL_LM",
)
# 1 - Enabling gradient checkpointing to reduce memory usage during fine-tuning
original_model.gradient_checkpointing_enable()
# 2 - Using the prepare_model_for_kbit_training method from PEFT
original_model = prepare_model_for_kbit_training(original_model)
peft_model = get_peft_model(original_model, config)
```

1. 创建了一个 `LoraConfig` 对象，该对象包含了 LoRA 训练所需的配置信息：

- `r=32` 设置了 LoRA 秩 (rank) 为 32，这个值决定了低秩矩阵的大小，影响模型的表达能力和计算成本。随着 rank 的增加资源成本也会增加，如果 rank 为 full rank 就等价于 full fine-tuning，也就是全量微调。
- `lora_alpha=16`: 设置了权重缩放因子 alpha 为 16，这个值决定了 LoRA 适配器中学习到的权重应该被放大的程度。
- `target_modules`: 指定了要在模型中插入 LoRA 适配器的模块，也就是需要训练的参数。通常是自注意力机制中的 `q_proj` (query matrix)、`k_proj` (key matrix)、`v_proj` (value matrix) 和 `o_proj` (Output Projection)。也可以优化其中的几个模块，这些都可以灵活配置。
- `bias="none"`: 表示在 LoRA 适配器中不使用偏置项，不作为训练参数。
- `lora_dropout=0.01`: 设置了在 LoRA 适配器中使用的 dropout 比率为 1%，这是为了防止过拟合的常规做法。
- `task_type="CAUSAL_LM"`: 指定了任务类型为因果语言模型 (CAUSAL_LM)，这意味着模型将被训练来生成文本，即每个时刻的输出只依赖于之前的输入。
- 代码中启用了梯度检查点 (gradient checkpointing)，这是一种减少在微调过程中内存使用的技术。这通过调用 `original_model.gradient_checkpointing_enable()` 实现。我们知道在模型训练时，我们通常需要将所有前向传播的激活值保存下来以在模型进行反向传播的时候使用，但是这样就会非常占用模型显存。当然我们也可以不保存激活值，而是在计算梯度时重新计算，但是这样虽然减少了缓存占用，但是却增大了计算量，减慢了训练速度。梯度检查点就是一个介于全不丢和全丢弃的一个这样的技术方案，即只丢弃部分梯度值。

- 第 2 步使用 `prepare_model_for_kbit_training` 方法来准备原始模型进行 k-bit 训练，涉及量化等技术来进一步减少模型微调时的资源消耗。
- 最后，调用 `get_peft_model` 函数，将原始模型和 LoRA 配置传入，以获取一个准备好进行 PEFT 训练的模型实例，这个实例被赋值给 `peft_model` 变量。

finetune模型

【cell12】

```
output_dir = './peft-dialogue-summary-training/final-checkpoint'
import transformers
peft_training_args = TrainingArguments(
    output_dir = output_dir,
    warmup_steps=1,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=1,
    max_steps=2000,
    learning_rate=2e-4,
    optim="paged_adamw_8bit",
    logging_steps=100,
    logging_dir="./logs",
    save_strategy="steps",
    save_steps=100,
    evaluation_strategy="steps",
    eval_steps=100,
    do_eval=True,
    gradient_checkpointing=True,
    report_to="none",
    overwrite_output_dir = 'True',
    group_by_length=True,
)
peft_model.config.use_cache = False
peft_trainer = transformers.Trainer(
    model=peft_model,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    args=peft_training_args,
    data_collator=transformers.DataCollatorForLanguageModeling(tokenizer,
mlm=False),
)
```

代码的解释：

- `output_dir` 定义了模型训练过程中保存输出文件的目录。
- `TrainingArguments` 是 `transformers` 库中的一个类，用于定义训练过程中的各种参数：
 - `output_dir`: 输出目录，训练过程中生成的所有文件将保存在这里。
 - `warmup_steps=1`: 指定在训练开始时的预热步数，预热期间学习率会逐渐增加。学习率先增

加，后减少。

- `per_device_train_batch_size=1`: 每个设备上的 mini-batch 大小。
 - `gradient_accumulation_steps=1`: 梯度累积步数，用于在更新模型之前累积多个批次的梯度，有助于模拟更大的批次大小。
 - `max_steps=2000`: 训练的最大步数。
 - `learning_rate=2e-4`: 学习率。
 - `optim="paged_adamw_8bit"`: 使用的优化器，这里指定了一个 8 位的AdamW 优化器。
 - `logging_steps=100`: 每 100 步记录一次日志。
 - `logging_dir`: 日志文件的保存目录。
 - `save_strategy="steps"`: 保存模型的策略，这里设置为按步数保存。
 - `save_steps=100`: 每 100 步保存一次模型。
 - `evaluation_strategy="steps"`: 评估模型的策略，这里设置为按步数评估。
 - `eval_steps=25`: 每 25 步进行一次评估。
 - `do_eval=True`: 是否在训练期间进行评估。
 - `gradient_checkpointing=True`: 是否启用梯度检查点。
 - `report_to="none"`: 不报告到任何平台，这通常用于避免上传到 WandB 或其他平台。
 - `overwrite_output_dir=True`: 如果输出目录已存在，是否覆盖。
 - `group_by_length=True`: 将类似长度的样本分组在一起，可以减少填充，提高训练效率。
3. `peft_model.config.use_cache = False`: 这行代码禁用了模型的缓存机制，这在某些生成任务中是有用的，但在训练时通常不需要。
4. 最后定义了 `peft_trainer`，其实就可以开始 fine-tune 模型了。

Finetune模型和评估

通过调用`train()`就可以进行微调训练。

【cell13】

```
peft_trainer.train()
```

如果报显存不够的问题，可以重启一下kernel，清除已占用的GPU显存，再从头运行一遍，保证没有额外的变量占用显存。

训练时的进度条和loss会自动打印：

Step	Training Loss	Validation Loss
100	1.509500	1.494385
200	1.372700	1.473312
300	1.375100	1.469163
400	1.358100	1.457793
500	1.367900	1.464802
600	1.286100	1.463337
700	1.306100	1.452105
800	1.306600	1.444739
900	1.361700	1.443314
1000	1.365100	1.439729
1100	1.337600	1.433075
1200	1.345900	1.432706
1300	1.330000	1.429296
1400	1.354500	1.428941
1500	1.320100	1.426169
1600	1.320000	1.424107

```
: TrainOutput(global_step=2000, training_loss=1.3488163146972656, metrics={'train_runtime': 2828.0835, 'train_
_samples_per_second': 0.707, 'train_steps_per_second': 0.707, 'total_flos': 1.2788117601509376e+16, 'train_
_loss': 1.3488163146972656, 'epoch': 1.0})
```

那么，如何知道训练的模型中训练了多少参数呢？我们可以使用下面的代码来统计：

```
def print_number_of_trainable_model_parameters(model):
    trainable_model_params = 0
    all_model_params = 0
    for _, param in model.named_parameters():
        all_model_params += param.numel()
        if param.requires_grad:
            trainable_model_params += param.numel()
    return f"trainable model parameters: {trainable_model_params}\n \
        all model parameters: {all_model_params}\n \
        percentage of trainable model parameters: \
        {100 * trainable_model_params / all_model_params:.2f}%"
```

调用即可。

```
print_number_of_trainable_model_parameters(original_model)
```

训练完成之后，可以把不用的模型参数等释放掉。

【cell14】

```
# Free memory
del original_model
del peft_trainer
torch.cuda.empty_cache()
```

Finetune 结束之后，你可以在你的项目目录下看到checkpoint，这些是通过 lora 模型的学习得出来的。为了能评估模型，我们首先需要做参数的合并，也就是 参数(base 模型) + 参数 (lora) 。

【cell15】

```
compute_dtype = getattr(torch, "float16")

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=compute_dtype,
    bnb_4bit_use_double_quant=True,
)
model_path = "/root/autodl-tmp/LLM-Research/Meta-Llama-3-8B-Instruct/"
base_model = AutoModelForCausalLM.from_pretrained(
    model_path,
    torch_dtype=compute_dtype,
    device_map={"": 0},
    quantization_config=quant_config
)
```

导入base model，和之前一样。

【cell16】

```
eval_tokenizer = AutoTokenizer.from_pretrained(model_path, add_bos_token=True,
trust_remote_code=True, use_fast=False)
eval_tokenizer.pad_token = eval_tokenizer.eos_token
```

设置用于推理的tokenizer。

【cell17】

```

from peft import PeftModel
ft_model = PeftModel.from_pretrained(
    base_model,
    "./peft-dialogue-summary-training/final-checkpoint/checkpoint-500",
    torch_dtype=torch.float16,
    is_trainable=False
)

```

导入lora部分参数，并且和base模型合并。

导入进来以后我们可以再查看一下显存的占用。

```

]: !nvidia-smi

```

huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...

To disable this warning, you can either:

- Avoid using `tokenizers` before the fork if possible
- Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

Mon Jun 17 22:57:09 2024

NVIDIA-SMI 550.54.14		Driver Version: 550.54.14		CUDA Version: 12.4	
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A
Fan	Temp		Pwr:Usage/Cap		Memory-Usage
					Volatil Uncorr. ECC
					GPU-Util Compute M.
					MIG M.
0	NVIDIA GeForce RTX 3090	On	00000000:4F:00.0 Off		N/A
30%	27C	P2	114W / 350W	14800MiB / 24576MiB	0%
					Default
					N/A

Processes:							GPU Memory
GPU	GI	CI	PID	Type	Process name		Usage
ID	ID						

可以看到现在的占用是14G左右。

【cell18】

```

%%time
index = 10

prompt = dataset['test'][index]['dialogue']
summary = dataset['test'][index]['summary']
formatted_prompt = f"Instruct: Summarize the following conversation.\nInput:
{prompt}\nOutput:\n"
res = gen(ft_model,formatted_prompt,100,)
#print(res[0])
output = res[0].split('Output:\n')[1]
dash_line = '-'.join(' ' for x in range(100))
print(dash_line)
print(f'INPUT PROMPT:\n{formatted_prompt}')
print(dash_line)
print(f'BASELINE HUMAN SUMMARY:\n{summary}\n')
print(dash_line)

```

```
print(f'PEFT MODEL GENERATION:\n{output}')
```

将dataset选择index序号的数据，包装为模型需要的格式，输入模型进行预测。我们一起来看一下预测结果：

```
-----
INPUT PROMPT:
Instruct: Summarize the following conversation.
Input:#Person1#: Happy Birthday, this is for you, Brian.
#Person2#: I'm so happy you remember, please come in and enjoy the party. Everyone's here, I'm sure you have a good time.
#Person1#: Brian, may I have a pleasure to have a dance with you?
#Person2#: Ok.
#Person1#: This is really wonderful party.
#Person2#: Yes, you are always popular with everyone. and you look very pretty today.
#Person1#: Thanks, that's very kind of you to say. I hope my necklace goes with my dress, and they both make me look good I feel.
#Person2#: You look great, you are absolutely glowing.
#Person1#: Thanks, this is a fine party. We should have a drink together to celebrate your birthday
Output:

-----
BASELINE HUMAN SUMMARY:
#Person1# attends Brian's birthday party. Brian thinks #Person1# looks great and charming.

-----
PEFT MODEL GENERATION:
#Person1# gives Brian a birthday gift and invites him to dance. Brian praises #Person1#'s beauty. #Person1# thanks Brian and wishes him a happy birthday. They both have a drink together to celebrate his birthday. They have a good time together. #Person1#'s necklace and dress make her look good. #Person1# feels happy. Brian praises #Person1#'s beauty. #Person1# thanks Brian. They both have a good time together. #Person1#'s necklace and dress make her look good. #Person1# feels happy. #Person2#'s praise makes #Person1# feel happy. They both have a good time together. #Person1#'s necklace and dress make her look good. #Person1# feels happy. #Person2#'s praise makes #Person1# feel happy. They both have a good time together. #Person1#'s necklace and dress make her look good. #Person1#
CPU times: user 20.5 s, sys: 155 ms, total: 20.7 s
Wall time: 20.7 s
-----
```

大家可以尝试通过调整生成时的**Temperature**、**Top-k Sampling**、**Beam Search**等参数来调整生成文本的随机性，也可以load不同的checkpoint来观察结果。

QLoRA 微调过程中遇到的问题及其解决方案

一般来说，QLoRA微调过程中可能会遇到的问题包括但不限于：

1. **量化精度问题**：QLoRA通过量化技术减少内存使用，但量化可能会引入精度损失，影响模型性能，损失多少和数据集相关，在实际应用中需要进行业务指标的测评。当算力不充足时，就需要平衡模型性能要求和算力，如果损失的性能在可接受范围内就没有问题。
2. **训练时间增加**：由于QLoRA在训练过程中引入了额外的量化和去量化步骤，可能会导致训练时间相对于未量化的LoRA方法有所增加。在这篇[blog](#)中做了实验对比：

We can see that QLoRA decreases the memory requirements by almost 6 GB. However, the tradeoff is a 30% slower training time, which is to be expected due to the additional quantization and dequantization steps.”

3. **硬件要求**：尽管QLoRA可以在较小的GPU内存上进行训练，但仍然需要特定的硬件支持，例如具备足够VRAM的GPU。

4. **模型微调效果**：QLoRA微调可能需要针对特定任务调整超参数，如低秩矩阵的秩（ r ）和缩放系数（ α ），以达到最佳效果。在这篇[blog](#)中做了实验对比：

	TruthfulQA MC1	TruthfulQA MC2	Arithmetic 2ds	Arithmetic 4ds	BLiMP Causative	MMLU Global Facts
Llama 2 7B base	0.2534	0.3967	0.508	0.637	0.787	0.32
AdamW QLoRA (nf4)	0.2803	0.4139	0.4225	0.006	0.783	0.23
AdamW + QLoRA + scheduler	0.2815	0.4228	0.182	0.001	0.783	0.27
All-layer QLoRA	0.3023	0.4409	0.51	0.028	0.788	0.26
QLoRA $r=16, \alpha=16$	0.3011	0.4338	0.529	0.0825	0.76	0.24
QLoRA $r=16, \alpha=32$	0.3158	0.4663	0.62	0.7005	0.757	0.27
QLoRA $r=32, \alpha=64$	0.2913	0.4362	0.5435	0.1265	0.762	0.28
QLoRA $r=32, \alpha=1$	0.2827	0.4111	0.512	0.4185	0.768	0.25
QLoRA $r=64, \alpha=128$	0.3035	0.4446	0.566	0.1255	0.765	0.29
QLoRA $r=64, \alpha=1$	0.284	0.4126	0.509	0.3735	0.766	0.24
QLoRA $r=128, \alpha=256$	0.3048	0.4548	0.5625	0.066	0.747	0.29
QLoRA $r=128, \alpha=1$	0.2864	0.4122	0.5095	0.3855	0.759	0.25
QLoRA $r=256, \alpha=512$	0.3035	0.4664	0.8135	0.3025	0.746	0.32
QLoRA $r=256, \alpha=1$	0.2827	0.411	0.509	0.3665	0.758	0.24

5. **如何避免过拟合**：一般来说，较大的 r 更可能导致过拟合，因为 r 决定着可训练参数的数量。如果模型存在过拟合问题，首先要考虑降低 r 值或增加数据集大小。此外，可以尝试增加AdamW或SGD优化器的权重衰减率，或者增加LoRA层的dropout值。
6. **内存峰值处理**：尽管QLoRA使用分页优化器来处理内存峰值，但在处理长序列长度的小批量数据时，内存管理仍然是一个挑战。
7. **推理速度**：QLoRA的推理速度可能受到量化位数的限制，特别是当使用4位量化时，推理速度可能会变慢。原因如下：
- **硬件支持**：并非所有的硬件都支持低位数量化的张量运算。如果硬件不支持4位量化的原生操作，那么执行这些操作可能需要额外的转换步骤，这会增加推理时的延迟。
 - **软件实现**：低位数量化可能需要特殊的软件实现和优化，以确保量化模型能够高效运行。如果软件框架没有针对4位量化进行优化，推理速度可能会受到影响。
 - **运算复杂性**：在某些情况下，使用低位数量化可能会增加运算的复杂性。例如，量化可能需要在运行时进行额外的舍入或零点校准操作，这些操作可能会降低推理速度。
 - **精度损失**：虽然QLoRA旨在保持性能，但量化可能会引入一定程度的精度损失。为了补偿这种损失，模型可能需要采用更复杂的算法或额外的处理步骤，这可能会影响推理速度。
 - **矩阵乘法效率**：在神经网络中，大量的计算是通过矩阵乘法完成的。量化可能会影响矩阵乘法的效率，尤其是在量化位数较低时，可能需要特定的优化技术来保持乘法操作的速度。
8. **模型部署限制**：量化后的模型可能需要特定的部署环境或额外的适配工作，以便在不同的平台上运行。
9. **特定实现的不稳定性**：在某些情况下，使用特定的量化配置（例如 `bnb_4bit_compute_type='fp16'`）可能会导致不稳定性。
- **数值精度限制**：4位量化大幅度减少了数值的表示范围和精度。这种低精度表示可能会导致数值计算中的累积误差，从而影响模型的稳定性和性能。
 - **数值稳定性问题**：在浮点运算中，尤其是在使用半精度（FP16）时，小的数值可能无法被准确表示，这可能导致数值稳定性问题，比如上溢（overflow）或下溢（underflow）。
 - **量化噪声**：量化过程本身会引入噪声，因为权重和激活被近似为离散值。这种噪声在4位量化中尤为显著，可能会对模型的稳定性产生负面影响。
 - **特定硬件的实现问题**：如果硬件对4位量化的支持不完善，或者软件框架对这种低精度量化的

实现存在缺陷，可能会导致运行时错误或不稳定的行为。

- **梯度流动问题**：在训练过程中，量化可能会影响梯度的流动，导致优化过程中的不稳定性。例如，量化可能会导致梯度的消失或爆炸问题。

总的来说，Qlora可以有效地对模型量化，节约GPU使用资源，但是同学们在实际使用过程中要注意这些可能会出现的问题。